Your multi-model AI MVP

A beginner-friendly build and validation roadmap for VC and accelerator review.

HMOS + IdeoLect

One product direction

Apna AI platform: multiple models ko orchestration, private project context, controlled tools aur independent validation ke saath jodna. Focus ek useful, repeatable MVP workflow par hai.

549 / 549

RECORDED ENGINEERING RESULT
Software tests passed at the reviewed HMOS commit. Yeh model-accuracy score nahi hai.

46% minimum

MODEL TASK-SUCCESS TARGET
Defined private benchmark par target. Actual live-model score abhi UNMEASURED hai.

100% pass

DEFINED ACCEPTANCE GOAL
Versioned release checks ke liye. Iska matlab har possible answer ya attack par 100% success nahi hai.

Up to 1M

CONDITIONAL CONTEXT STRETCH
Selected model/API ki actual support, quality, privacy aur cost verify karne ke baad.

Scope: beginner MVP build → measured 46% or higher → VC/accelerator review. Remaining roadmap preserved, with a short continuation map.

Pages 2–4: current position and score. Pages 5–10: build and validation. Page 11: exact MVP stop line. Page 12: retained research map. Pages 13–14: investor packet and sources.

One small loop. Real evidence.

MVP mein wire karna hai: authorized account → private project → model/tool → independent verification → saved checkpoint.

Track	Current position	MVP role
HMOS	Tested application / prototype components	Architecture review, project context, agent workflow and validation
IdeoLect Core	Pre-alpha; implemented security slices	Scoped authority, local data controls and bounded contained execution
IdeoLect Frontier	Planned research track; repository reported empty	Research direction to describe clearly; no delivered model capability assumed

Start with this user job

“Mere selected private repo ka ek bug ya architecture question samjhao, relevant file references do, aur jo verify nahi hua usko UNKNOWN bolo.” Pehla demo owned code aur safe fixtures par rakho.

1. Identity → context

Server-verified account + project access. Phir sirf allowed files se retrieval.

2. Model → bounded tool

Pinned model route + owned read-only tool. Server enforces allowed action, token/cost budget and timeout.

3. Verify → checkpoint

Independent answer/citation verdict; selected project ka checkpoint save. Sensitive effects need approval.

4. Exercise the loop

One legitimate success, another-user denial, provider failure, revoked session and restart/recovery. Observe real effects.

Yeh controlled enterprise-style MVP path hai; full enterprise certification nahi. Existing licensed models se own orchestration valid hai. Trained foundation-model weights aur neural MoE training separate deliverables hain. Optional second-model review ko gain measure hone par add karo.

Built slices. Open milestones.

Original HMOS roadmap: S00–S16 + 12 MAX-17 pillars + 14 MAX-18 pillars = 43 items. Inko rename ya remove nahi kiya gaya.

Roadmap connection	Recorded local position	Still open
MAX-17.02 / 17.12; MAX-18.08 / 18.09	PARTIAL: account/project-bound memory, persistence and stale-session refusal tests	Production auth/UI integration; backend scale; deletion, retention and full recovery
S06 / S08 / S09; MAX-18.01 / 18.02	PARTIAL: culprit hypotheses plus reproduction, intervention, control and replay harness	Real-model integration, independent quality baseline and whole-stage acceptance
MAX-17.05; MAX-18.02 / 18.05	PARTIAL: source/duplicate checks, synthetic-data validation and measured-candidate ranking hooks	Approved real corpus, actual generated/validated data, routing gain and held-out score
S10; MAX-17.11 / 17.12	PARTIAL: bounded Docker lifecycle and confirmed cleanup tests	Full production policy, sandbox service and operational acceptance
MAX-17.01 / 17.02; MAX-18.11	PENDING: reviewed validation marks production auth and account/session quotas unimplemented	Server verification, ownership checks and IP-independent account budgets
IdeoLect Core P01–P12	P01–P06 partial; P07–P12 planned in current source	No complete end-to-end phase or overall completion percentage established

Growth is visible. Quality still needs a score.

Actual additions: scoped memory, culprit-checking harness, synthetic validation and Docker cleanup hardening. These deepen the implementation. Live-model quality is still UNMEASURED. Whole stages/pillars remain open until their own gates pass; mappings above are connections, not completion certificates.

HMOS evidence/latest.json is older, has skipped/not-run checks, and has no per-roadmap-item PASS records. The newer 549/549 local record is a separate bounded result. This PDF reconciles those boundaries; it does not rewrite the roadmap or mark all other items complete.

46% minimum. Measure it first.

Accuracy tab useful proof banti hai jab task, denominator, grading aur failure handling pehle fix hon.

Primary MVP metric

Overall task success = fully correct tasks ÷ all assigned tasks.
Example benchmark size: 300 private tasks. Minimum target 46% = **at least 138 fully correct tasks**.
Timeout, tool failure, abstention aur UNKNOWN primary score mein success nahi hain.

Metric	Meaning	Example only — not a measured result
Overall success	Correct / all tasks	120 / 300 = 40.0%
Attempted-answer precision	Correct / answered tasks	120 / 130 = 92.3%
Answer coverage	Answered / all tasks	130 / 300 = 43.3%
Acceptance pass rate	Passed required checks / required checks	Goal: 100 / 100 defined checks

“Overall” ka boundary

Yeh score aapke disclosed task mix ka hai: for example repo Q&A;, bug diagnosis, ya bounded code fixes. Isse general intelligence ka percentage, universal frontier ranking, ya competitor comparison infer nahi hota.

Grade before you optimize

Har task ke expected facts, valid citations aur functional checks define karo. Code fixes ko hidden assertions + regression tests se grade karo. Semantic answers ke liye fixed rubric aur independent human review use karo; model ki self-rating proof nahi hai.

Build progress is a different metric: earlier “60% built” is not a measured AI-accuracy baseline. More roadmap scope cannot convert into a score. Below 46%, the quality gate stays open. Report higher actual results and uncertainty honestly; 100% checks and task success use different denominators.

Build the smallest useful loop.

Yeh PDF checklist numbers hain, original roadmap ids nahi. Existing roadmap dependencies aur working modules preserve karo.

1 Ek task aur clear rubric choose karo

Ek owned repository, ek question family aur 10–20 approved sample questions choose karo. Expected answer aur allowed source files likho.

Proof to save: Task brief, grading rubric, approved source list and a short demo script.

2 Ek real model connect karo

Documented model route se real response lo. Model/version, input/output tokens, errors aur actual cost record karo. Unknown cost ko null/unknown rakho.

Proof to save: One successful run, one failed-provider run, model id, response and usage log.

3 Account aur selected project bind karo

Server par session verify karo, project ownership check karo, phir files retrieve karo. Display name ya prompt se owner identity accept mat karo.

Proof to save: Allowed-user access, another-user denial, expired-session denial and account-switch tests.

4 Short context + grounded answer chalao

8K–32K input budget se start karo, only relevant chunks lo, file/revision citations do. Unsupported answers ko review/UNKNOWN path par bhejo.

Proof to save: Question → retrieved files → answer → independent verdict ka complete trace.

Local setup example

```
nvm use
npm ci
npm run check
npm run dev
```

Commands HMOS repository root ke liye hain. Pehle AGENTS.md aur README padho. Model keys server-side environment mein rakho; logs, browser bundle aur PDF mein keys mat daalo.

Turn the loop into MVP proof.

Pehle quality measure karo. Phir complexity tabhi add karo jab measurable benefit mile.

5 Evaluation set aur splits freeze karo

Development examples, tuning set, private holdout aur release-gate cases alag rakho. Source/project families ke basis par split karke near-duplicates aur answer leakage kam karo.

Proof to save: Versioned task ids, source hashes, split manifest and independent grading rules.

6 Baseline run karo

Single-model + basic retrieval ko baseline banao. Failed cases ko missing evidence, wrong reasoning, broken tool, stale memory aur policy error mein classify karo.

Proof to save: Correct/all counts, coverage, error categories, latency, token usage and measured/unknown cost.

7 Small multi-model routing add karo

Optional second model ko review ya alternate candidate ke liye add karo. Same development set par compare karo. Safe mandatory gates pass hone ke baad hi candidates rank karo.

Proof to save: Baseline vs candidate comparison, paired wins/losses, extra calls and total cost.

8 Freeze, validate, then record the demo

Final configuration lock karo; private holdout grade karo; required acceptance checks run karo. One controlled workflow ka repeatable demo aur project checkpoint save karo.

Proof to save: Versioned score report, complete pass/fail ledger, demo video and known-limitations note.

How to investigate a real culprit

Competing hypotheses → reproduce the symptom → targeted intervention → negative control → regression → fresh replay. Model explanation se zyada independent execution evidence matter karti hai. Ambiguous ya missing evidence ko verified culprit mat banao.

Breadth + depth. Every slice.

Aapke “deep hariant” ko yahan deep logic / invariant-depth method ke roop mein use kiya hai. Yeh kisi model ka measured capability label nahi hai.

Build area	Depth: what must remain true	Breadth: what to vary
Identity / memory	Only authenticated account + selected project can authorize retrieval	Same names, other accounts, expiry, revocation, deletion, stale cache and restore
Data / context	Sources are allowed, current, cited and separated from instructions	Revisions, source positions, languages, missing evidence, conflicting facts and injection
Reasoning / culprit	A cause requires independent reproduction, intervention, controls and regression	Alternative causes, misleading symptoms, wrong patch, flaky tests and ambiguous outcomes
Tools / recovery	Allowed effect stays inside authority and budget; unknown outcome remains unknown	Lost reply, crash, duplicate dispatch, late worker, cancellation and legitimate success
Models / evaluation	Model cannot certify itself or change the private grading contract	Task families, difficulty, routing, abstention, token budgets and provider failures

One loop, applied everywhere

Choose one real build slice → define invariant and assumptions → create valid/invalid/recovery/benign cases → observe with an independent oracle → classify the gap → change code → rerun affected evidence. Then widen the supported interface.

Synthetic data has one job

Uncovered behavior aur failure schedules discover karna. Accepted examples need approved sources, deduplication and independent validation. More paraphrases are not more architectural coverage. Inconclusive stays UNKNOWN; useful allowed work must still succeed.

Use this method for the first MVP and the retained roadmap alike. A finite invariant proof covers its declared model; synthetic test coverage does not guarantee all real-world behavior.

Use better data, then more data.

Yeh suggested starting design hai. Data quantity ya model parameter count se score guarantee nahi hota.

Partition	Example size	Allowed use
Development / example pool	400	Source-grounded examples, prompt design and permitted training preparation
Tuning / development eval	200	Routing, retrieval, prompts and error analysis
Private final holdout	300	Frozen-config evaluation; never shown to generators or models as examples
Acceptance / adversarial suite	100	Defined expected allow/deny, recovery and release behavior

Synthetic data acceptance

Approved source + permission/license metadata + source hash save karo. Generator ko sirf development sources do. Examples ko exact/near-duplicate checks aur independent deterministic tests ya human validation ke baad accept karo. Model approving its own answer is not independent validation.

Keep the comparison fair

Baseline aur candidate ko same task-set version, same grading, same tool policy aur disclosed token/cost budgets par compare karo. Tuning ke liye 200-task set use karo. Final holdout feedback se next build tune kiya to us set ko retire karo; next final gate ke liye fresh held-out families aur new version use karo.

Small gains can be noise

Worked example: 120/300 correct = 40%. A 95% Wilson interval is approximately **34.6%–45.6%**. Yeh actual project result nahi hai. A small score increase alone does not prove an upgrade; paired errors, sample size and repeatability bhi report karo.

Minimum report fields: task-set version, N, correct, failed/unknown, attempted, model ids, source revision, context size, latency, tokens, cost and grader version. Current project live-model score remains **UNMEASURED**.

A complete gate. A finite claim.

Neeche proposed 100-case MVP acceptance matrix hai. Is suite ko implement/run karna baaki hai.

Check family	Cases	What the suite should include
Authentication / ownership	16	Valid access plus expired, forged and missing-session denial
Project / cache separation	16	Same display names, account switch, cross-project reads and scoped caches
Memory / injection boundary	12	Old instructions treated as data; stale/foreign records rejected
Tools / containment	12	Allowed action, forbidden path, denied egress and inspected isolation
Timeout / quotas / cleanup	12	Budget exhaustion, provider failure, revoked run and confirmed cleanup
Evidence / reporting	12	Empty tests, fabricated pass, unsupported citation and UNKNOWN handling
Legitimate useful workflows	12	Permitted work completes; deny-everything is not a valid pass
Recovery / replay	8	Restart, idempotency, duplicate dispatch and current-state checks

Release rule

Required cases fixed before the run. **100 passed / 100 required, with zero skipped or unknown**, is the goal for this suite. Any required failure blocks release. Fresh human review remains part of the controlled MVP workflow.

100/100 means this version passed these cases. It is not 100% model accuracy or proof against every attack. The recorded 549/549 HMOS software run is separate engineering evidence; it does not certify this proposed 100-case suite or production authentication.

Start short. Earn the 1M claim.

Context window, data-store size, output length aur model quality alag cheezein hain.

Stage	Purpose	Evidence before moving on
8K–32K	First useful MVP workflow	Correct retrieval, citations, account/project filters and stable cost
128K	Larger selected-project tasks	Quality vs short-context baseline; beginning/middle/end evidence tests
Up to 1M	Optional MVP stretch experiment	Exact model/API support, request/output limits, quality, privacy, latency and budget verified

Illustrative shared-window budget

If a selected model has a shared **1,000,000-token** window:
952,000 selected-input + 16,000 planned-output + 8,000 system/tool budget + 24,000 safety margin = 1,000,000.
Only an example: actual API accounting, output caps and request limits must be checked for the exact model version.

Test effective context, not just request acceptance

Put relevant evidence at the beginning, middle and end. Test multi-file joins, conflicting revisions, irrelevant padding and injected instructions. Compare answers and citations against a short-context retrieval baseline on matched tasks.

Input ≠ output; memory ≠ the prompt

1M context does not promise 1M output in a single response. Long deliverables can be produced as checked sections/files with checkpoints. Stored project data should stay outside the prompt until an authorized query retrieves relevant pieces.

Current local HMOS memory has a 32 MiB opening limit; that is a storage/index boundary, not a context-token claim. Million/trillion-record search, local hardware capacity and 1M-token model support require separate proof. If the longer input makes quality or cost worse, keep the smaller effective context.

Pause here. Present the proof.

Stop kisi arbitrary stage number par nahi: below MVP evidence gates par. Full 43-item roadmap ko complete kehna required nahi hai.

Required MVP gate	Original roadmap connection	Exit evidence
One useful supported workflow	S00–S04; S14	Declared user job, boundaries, runnable demo and instructions
Private account / project path	MAX-17.01 / 17.02 / 17.11	Real server auth + ownership + tools checks before any private multi-user use
Grounded analysis + bounded repair	S06 / S08 / S09 / S10; MAX-18.01 / 18.04 / 18.08	Independent verdicts, UNKNOWN path; tested causal evidence for any verified-culprit claim
Data + model + context contract	MAX-17.04 / 17.05; MAX-18.02 / 18.05 / 18.09	Pinned models, allowed data, scoped memory and actual context benchmark
Quality, cost and release checks	S12; MAX-18.11	Measured 46% or higher; 100/100 required checks; latency/cost and failures disclosed
Reproducible investor packet	S15; selected S13 controls	Code/config/task-set versions, independent rerun, demo and limitation register

The pause rule

One small end-to-end loop works, task success is at least 46%, and all selected MVP gates have real evidence. Freeze that version, present it to VCs/accelerators, and retain the rest. Below 46%, continue development from measured failure categories and use a fresh final evaluation. Privacy/release blockers also keep the gate open.

These are bounded MVP slices, not full certifications of the linked pillars. Missing dependencies remain blockers. Without production auth, keep the prototype single-user/offline on owned fixtures. Optional 1M context and full enterprise delivery do not block a clearly scoped smaller MVP.

Keep the rest. Continue from proof.

80%+ is a future research target on a defined workload. This page preserves direction; the beginner build guide stops at the MVP gate.

Retained work	Original roadmap ids	What future evidence must improve
Deeper reasoning + broader task families	S06–S09; MAX-18.01–04	Fresh causal tests, calibration, independent grading and failures across useful tasks
Stronger data, retrieval and memory	MAX-17.05 / 17.12; MAX-18.08–10	Coverage, provenance, freshness, deletion, recovery and safe human escalation
Model lifecycle and adaptive compute	MAX-17.04; MAX-18.05–07 / 18.11	Measured quality gain per task, routing ablations, actual latency/cost and rollback
Production security and operations	S10–S13 / S16; MAX-17.01–03 / 17.06–11	Supply-chain integrity, enforced identity, observability, SLOs, restore and incident drills
Delivery and renewed assurance	S14–S15; MAX-18.12–14	Reproducible releases, API contracts, fresh evaluations and source-bound certification
IdeoLect security track	Finish open P01–P06 scope; retain P07–P12	Integrated authority, containment, intelligence, proof, remediation and delivery gates

A score is not roadmap completion

Illustrative 300-task benchmark: minimum MVP gate 46% = 138 correct; 80% = 240 correct. Add fresh held-out tests as capability grows and report them separately. Keep an anchored comparison where appropriate; a changed task mix needs a new version and cannot silently replace the old score.

Funding, more parameters, 1M context or finishing more pillars cannot guarantee 80%+. The route is a falsifiable hypothesis: source-grounded data + synthetic breadth + deep invariant logic + independent validation may improve measured task success. No automatic per-stage percentage increase is assigned.

Make the proof easy to inspect.

Show one repeatable workflow with honest boundaries. Engineering proof aur AI-quality proof ko separate rakho.

Available engineering evidence

HMOS: 549/549 recorded software tests; all 5 PR CI/Security checks passed; protected-file ratchet passed. Linked to exact reviewed code.

Quality evidence to collect

46% minimum benchmark target, defined acceptance results and effective-context tests. Actual live-model scores, pilot users and business traction are not supplied.

A five-minute demo outline

1. Introduce one user problem and the chosen task.
2. Open the authorized account/project and show its source revision.
3. Ask one question; show retrieved evidence and selected model route.
4. Show the independent verdict, one failure/UNKNOWN case and its handling.
5. Show the saved project checkpoint and the matching score report.

Attach these six items

Product one-pager • recorded demo • reproducible run instructions • versioned benchmark report • architecture/security boundaries • concise next-build milestones and resource needs.

A safe investor-facing sentence

“We are building a multi-model AI platform with private project context and evidence-based validation. Engineering components are tested. Our next MVP proof milestone is at least 46% overall success on a disclosed private task set, with all required release checks passing.”

Use live UI only for paths actually integrated. Engine fixtures, simulations and recorded runs should be visibly labelled. A larger context window or a high software-test pass rate is not a substitute for measured task quality.

Your final founder checklist

Technical MVP proof tak ka scope. Every result should point to a real run.

Record	Fill with evidence
Task-set / grader version	_____
Model ids / code revision	_____
Overall task success	___ / 300 = ___% (minimum target: 138 correct)
Acceptance suite	___ / 100; skipped ___; unknown ___
Context actually evaluated	_____ tokens; quality / cost report attached
Latency / tokens / actual cost	_____

Keep the first cloud run bounded

Platform credits aur limited tool subscriptions current founder-reported resources hain. Exact AWS service/credit eligibility, expiry, pricing aur quotas check karo. Daily account/session cost caps, max calls, concurrency and stop controls set karo. IP rate limiting can supplement account quotas; IP is not ownership.

Cost estimate: input tokens × input rate + billable output/reasoning tokens × output rate + compute/storage/network + retries. Provider units and cached-token rates verify karo. Unknown actual cost ko measured value ki tarah report mat karo.

Source notes — reviewed engineering context

1. HMOS PR #40 — merged implementation and CI checks
2. Recorded local validation — reviewed code f388f3a
3. HMOS capability boundaries and remaining integrations
4. Original HMOS roadmap — 43 unchanged items and dependencies
5. IdeoLect Core — current P01–P12 status
6. Invariant depth + synthetic breadth — combined engineering method

Source snapshot: 21 October 2026. Repository links may require authorized access. Targets, example dataset sizes, arithmetic examples and the 100-case matrix in this guide are planning material, not newly executed benchmarks. Production authentication/quotas and live-model quality still require their own validation.